

Объекты и методы объектов

Конспект по JavaScript

Объекты (Objects)

Объект (object) — способ описать характеристики чего-либо (пользователь, книга, товар) в виде набора пар «ключ: значение». Объекты ещё называют словарями (dictionaries): ключ — это термин, значение — его определение.

Создание объекта

Объект создаётся через фигурные скобки `{}` — это называется литерал объекта (object literal).

```
const book = {  
  title: "The Last Kingdom",  
  author: "Bernard Cornwell",  
  genres: ["historical prose", "adventure"],  
  public: true,  
  rating: 8.38,  
};
```

- Каждое свойство (property) — это пара ключ: значение.
- Ключ (key) — обычно строка, ещё называется именем свойства (property name).
- Значением может быть что угодно: примитив, массив, объект, булево значение (boolean), функция.
- Свойства разделяются запятой.

Вложенные свойства (nested properties)

Значением свойства может быть другой объект — так группируют связанные данные.

```
const user = {  
  name: "Jacques Gluke",  
  tag: "jgluke",  
  location: {  
    country: "Jamaica",  
    city: "Ocho Rios",  
  },  
  stats: {  
    followers: 5603,  
    views: 4827,  
    likes: 1308,  
  },  
};
```

Например, статистику (stats) или локацию (location) удобно хранить отдельным вложенным объектом — потом по этим данным легко искать и фильтровать.

Доступ к свойствам через точку (dot notation)

Синтаксис: `objectName.key`. Используется, когда имя свойства заранее известно.

```
const bookTitle = book.title;  
console.log(bookTitle); // "The Last Kingdom"  
  
const bookGenres = book.genres;  
console.log(bookGenres); // ["historical prose", "adventure"]  
  
const bookPrice = book.price;  
console.log(bookPrice); // undefined
```

Если свойства с таким ключом нет — вернётся `undefined`.

Доступ к вложенным свойствам

Обращения «через точку» выстраиваются в цепочку (chain):

```
const location = user.location;
console.log(location); // {country: "Jamaica", city: "Ocho Rios"}

const country = user.location.country;
console.log(country); // "Jamaica"
```

Если значение — массив (array), к элементам обращаемся через индекс в квадратных скобках, а к длине — через свойство `length`:

```
const hobbies = user.hobbies;
console.log(hobbies); // ["swimming", "music", "sci-fi"]

const firstHobby = user.hobbies[0];
console.log(firstHobby); // "swimming"

const numberOfHobbies = user.hobbies.length;
console.log(numberOfHobbies); // 3
```

Доступ через квадратные скобки (bracket notation)

Синтаксис: `objectName["key"]`. Используется реже — когда имя свойства заранее неизвестно и хранится в переменной.

```
const book = {
  title: "The Last Kingdom",
  author: "Bernard Cornwell",
  genres: ["historical prose", "adventure"],
  isPublic: true,
  rating: 8.38,
};

console.log(book.title); // "The Last Kingdom"
console.log(book["title"]); // "The Last Kingdom"

const propKey = "author";
console.log(book.propKey); // undefined
console.log(book[propKey]); // "Bernard Cornwell"
```

`book.propKey` → `undefined`, потому что в объекте нет свойства с ключом «**propKey**» (ищется буквальное имя).
`book[propKey]` → `"Bernard Cornwell"`, потому что `propKey` — переменная со значением «**author**», и это значение используется как ключ.

Критерий	Через точку (dot notation)	Через скобки (bracket notation)
Синтаксис	<code>object.key</code>	<code>object["key"]</code>
Когда используется	имя свойства известно заранее	имя свойства хранится в переменной или вычисляется
Если свойства нет	<code>undefined</code>	<code>undefined</code>

Изменение значений свойств

Обращаемся к свойству и присваиваем новое значение:

```
book.rating = 9;
book.isPublic = false;
book.genres.push("drama");

console.log(book.rating);    // 9
console.log(book.isPublic);  // false
console.log(book.genres);    // ["historical prose", "adventure", "drama"]
```

Добавление свойств

Если свойства с таким ключом не было — оно создастся автоматически (запись = добавление, если ключа ещё нет):

```
book.pageCount = 836;
book.originalLanguage = "en";
book.translations = ["ua", "ru"];
book.price = {
  hardcover: 39,
  softcover: 29,
};

console.log(book.pageCount);    // 836
console.log(book.originalLanguage); // "en"
console.log(book.translations); // ["ua", "ru"]
```

Короткие свойства (shorthand properties)

Если значение свойства берётся из переменной с таким же именем, как и само свойство — не нужно дублировать имя.

Громоздкий вариант:

```
const name = "Henry Sibola";
const age = 25;

const user = {
  name: name,
  age: age,
};
```

С shorthand properties:

```
const name = "Henry Sibola";
const age = 25;

const user = {
  name,
  age,
};

console.log(user.name); // "Henry Sibola"
console.log(user.age);  // 25
```

Достаточно указать только имя переменной — значение возьмётся из одноимённой переменной.

Вычисляемые свойства (computed properties)

Позволяют задать ключ (имя) свойства прямо при создании объекта, если он хранится в переменной или является результатом выражения — без создания объекта отдельным шагом.

Раньше приходилось так:

```
const propName = "name";
const user = {
  age: 25,
};

user[propName] = "Henry Sibola";
console.log(user.name); // "Henry Sibola"
```

C computed properties — сразу при объявлении:

```
const propName = "name";
const user = {
  age: 25,
  // ключ этого свойства будет взят из значения переменной propName
  [propName]: "Henry Sibola",
};

console.log(user.name); // "Henry Sibola"
```

Значением вычисляемого свойства (в квадратных скобках при объявлении объекта) может быть любое валидное выражение.

Перебор объекта (Object iteration)

Цикл for...in

Объект (object) — не итерируемая (iterable) сущность, в отличие от массива (array) или строки (string). Обычные циклы for и for...of для объектов не работают.

Для перебора объектов есть специальный цикл for...in — он перебирает ключи (keys) объекта.

```
for (key in object) {
  // инструкции
}
```

- Переменная key доступна только внутри тела цикла.
- На каждой итерации (iteration) в неё попадает очередной ключ (имя свойства).
- Чтобы получить значение свойства по этому ключу — используется синтаксис квадратных скобок (bracket notation).

```
const book = {
  title: "The Last Kingdom",
  author: "Bernard Cornwell",
  genres: ["historical prose", "adventure"],
  rating: 8.38,
};

for (const key in book) {
  console.log(key); // ключ
  console.log(book[key]); // значение свойства с этим ключом
}
```

Метод Object.keys()

Встроенный класс (built-in class) Object содержит полезные методы для работы с объектами.

Object.keys(object) — принимает объект и возвращает массив (array) ключей всех его свойств. Если свойств нет — вернётся пустой массив.

```
const book = {
  title: "The Last Kingdom",
  author: "Bernard Cornwell",
  genres: ["historical prose", "adventure"],
  rating: 8.38,
};
const keys = Object.keys(book);
console.log(keys); // ['title', 'author', 'genres', 'rating']
```

Комбинация `Object.keys()` + цикл `for...of` — удобная альтернатива `for...in`:

```
const book = {
  author: "Bernard Cornwell",
  genres: ["historical prose", "adventure"],
  rating: 8.38,
};
const keys = Object.keys(book);

for (const key of keys) {
  console.log(key);      // ключ
  console.log(book[key]); // значение свойства
}
```

Перебираем массив ключей объекта, и на каждой итерации по ключу достаём значение свойства.

Метод `Object.values()`

`Object.keys(object)` возвращает ключи (имена свойств), а `Object.values(object)` возвращает массив значений (`values`) этих свойств.

Если свойств нет — тоже вернёт пустой массив.

```
const book = {
  title: "The Last Kingdom",
  author: "Bernard Cornwell",
  rating: 8.38,
};
const keys = Object.keys(book);
console.log(keys); // ["title", "author", "rating"]

const values = Object.values(book);
console.log(values); // ["The Last Kingdom", "Bernard Cornwell", 8.38]
```

Массив значений тоже можно перебрать через `for...of` — например, чтобы посчитать сумму числовых значений.

Массив объектов (Array of objects)

Объекты (objects) группируют характеристики одной сущности. Чтобы описать группу однотипных сущностей (например, коллекцию книг), используется массив объектов (array of objects) — каждый элемент массива это отдельный объект.

```
const books = [
  {
    title: "The Last Kingdom",
    author: "Bernard Cornwell",
    rating: 8.38,
  },
  {
    title: "Beside Still Waters",
    author: "Robert Sheckley",
    rating: 8.51,
  },
  {
    title: "The Dream of a Ridiculous Man",
    author: "Fyodor Dostoevsky",
    rating: 7.75,
  }
];
```

Важно: все объекты в таком массиве обычно имеют одинаковый набор свойств (properties), но разные значения (values).

Для перебора массива объектов используется обычный цикл `for...of`. Доступ к свойствам каждого объекта — через точку (dot notation), так как имена свойств у всех объектов одинаковые:

```
for (const book of books) {
  console.log(book);      // объект книги
  console.log(book.title); // название
  console.log(book.author); // автор
  console.log(book.rating); // рейтинг
}
```

Поиск объекта по значению свойства

Частая задача — найти объект в массиве по значению его свойства (например, найти книгу по автору). Для этого нужно:

- перебрать массив в цикле;
- добавить условие (condition), при выполнении которого объект считается найденным.

```
const books = [
  { title: "The Last Kingdom", author: "Bernard Cornwell" },
  { title: "Beside Still Waters", author: "Robert Sheckley" },
  { title: "The Dream of a Ridiculous Man", author: "Fyodor Dostoevsky" }
];

const authorToSearchFor = "Robert Sheckley";

for (const book of books) {
  if (book.author === authorToSearchFor) {
    console.log(book);
    console.log(book.title);
    console.log(book.rating);
  }
}
```

Цикл перебирает объекты, пока не найдёт тот, где `book.author === authorToSearchFor` даёт `true`. Как только объект найден — выполняется тело `if`, и с найденным объектом можно работать дальше.

Коллекция значений свойства

Ещё одна типовая задача — собрать в отдельный массив все значения конкретного свойства из коллекции объектов (например, все названия книг или все рейтинги).

Порядок действий:

1. Создать новый пустой массив для значений.
2. Перебрать массив объектов циклом.
3. На каждой итерации добавить в новый массив значение нужного свойства.

Пример — собрать список всех названий книг:

```
const books = [
  { title: "The Last Kingdom", author: "Bernard Cornwell", rating: 8.2 },
  { title: "Beside Still Waters", author: "Robert Sheckley", rating: 9 },
  { title: "The Dream of a Ridiculous Man", author: "Fyodor Dostoevsky", rating: 6.8 }
];

const titles = [];

for (const book of books) {
  titles.push(book.title);
}

console.log(titles); // ["The Last Kingdom", "Beside Still Waters", "The Dream of a Ridiculous Man"]
```

Пример — посчитать средний рейтинг (average rating) коллекции: сложить все рейтинги и разделить сумму на количество книг.

```
const books = [
  { title: "The Last Kingdom", author: "Bernard Cornwell", rating: 8.2 },
  { title: "Beside Still Waters", author: "Robert Sheckley", rating: 9 },
  { title: "The Dream of a Ridiculous Man", author: "Fyodor Dostoevsky", rating: 6.8 }
];

let totalRating = 0;

for (const book of books) {
  totalRating += book.rating;
}

const averageRating = totalRating / books.length;
console.log(averageRating); // 8
```

Методы объекта (Object methods)

Зачем нужны методы

До сих пор объекты (objects) были просто хранилищами данных. Но что если нужны функции для работы с этими данными — например, добавление/удаление книги, и эти операции должны выполняться многократно?

Плохой вариант — независимые сущности без связи между собой:

```
// ❌ слабо связанные, независимые сущности
const books = ["The Last Kingdom", "Dream Guardian"];
function getBooks() {}
function addBook() {}
```

Лучше: объекты могут хранить не только данные, но и функции для работы с ними. Если значением свойства (property) является функция — такое свойство называется методом объекта (object method).

```
const obj = {
  method(value) {
    console.log(`I'm a method with ${value}!`);
  }
};

obj.method(5); // "I'm a method with 5!"
obj.method(10); // "I'm a method with 10!"
```

Метод — обычная функция, объявленная как свойство объекта. Вызывается через точку (dot notation), как и обычное свойство.

Объекты, которые объединяют данные и методы для работы с ними, называют «моделями» (models):

```
//  логически и синтаксически связанные сущности
const bookShelf = {
  books: ["The Last Kingdom", "Dream Guardian"],
  getBooks() {
    return "Returning all books";
  },
  addBook(bookName) {
    return `Adding book ${bookName}`;
  },
};

bookShelf.getBooks(); // "Returning all books"
bookShelf.addBook("New book 1"); // "Adding book New book 1"
```

Доступ к свойствам объекта через this

Внутри метода для обращения к своему же объекту используется не имя переменной, а ключевое слово `this`. `this` — это контекст выполнения (execution context) функции: ссылка на объект, который стоит перед точкой при вызове метода.

```
const bookShelf = {
  books: ["The Last Kingdom", "The Mist"],
  getBooks() {
    console.log(this);
  },
};

bookShelf.getBooks(); // {books: [...], getBooks: f}
```

Доступ к свойствам через `this`:

```
const bookShelf = {
  books: ["The Last Kingdom", "The Mist"],
  getBooks() {
    return this.books;
  }
};

console.log(bookShelf.getBooks()); // ["The Last Kingdom", "The Mist"]
```

Почему не имя объекта напрямую? Имя объекта — ненадёжная вещь: методы можно скопировать в другой объект с другим именем, а иногда имя объекта заранее вообще неизвестно. `this` гарантирует, что метод работает именно с тем объектом, который его вызвал.

Изменение по ссылке (change by reference)

Массив (array), хранящийся в свойстве, можно менять по ссылке (by reference) прямо через обращение к свойству:

```
const bookShelf = {
  books: ["The Last Kingdom"],
};

bookShelf.books.push("The Mist");
console.log(bookShelf.books); // ["The Last Kingdom", "The Mist"]
```

Через `this.books` внутри метода — ссылка на тот же массив, поэтому его тоже можно менять методами массива (например, `push()`):

```
const bookShelf = {
  books: ["The Last Kingdom"],
  getBooks() {
    return this.books;
  },
  addBook(bookName) {
    this.books.push(bookName);
  }
};

console.log(bookShelf.getBooks()); // ["The Last Kingdom"]
bookShelf.addBook("The Mist");
bookShelf.addBook("Dream Guardian");
console.log(bookShelf.getBooks()); // ["The Last Kingdom", "The Mist", "Dream Guardian"]
```

Методу `addBook` ничего не нужно возвращать (`return`) — он просто меняет массив по ссылке, и данные во внешнем свойстве `books` меняются сами.

Массив объектов внутри модели

Чаще всего в свойстве хранят не строки, а объекты с несколькими характеристиками (например, название книги и рейтинг):

```
const bookShelf = {
  books: [
    { title: "The Last Kingdom", rating: 8 },
    { title: "The Mist", rating: 6 }
  ],
  getBooks() {
    return this.books;
  }
};
```

Метод `addBook` теперь принимает объект книги целиком:

```
const bookShelf = {
  books: [
    { title: "The Last Kingdom", rating: 8 },
    { title: "The Mist", rating: 6 }
  ],
  getBooks() {
    return this.books;
  },
  addBook(newBook) {
    this.books.push(newBook);
  }
};

bookShelf.addBook({ title: "Dream Guardian", rating: 9 });
```

Пример метода, который считает средний рейтинг (average rating) книг:

```
const bookShelf = {
  books: [
    { title: "The Last Kingdom", rating: 8 },
    { title: "The Mist", rating: 6 },
  ],
  getAverageRating() {
    let totalRating = 0;

    for (const book of this.books) {
      totalRating += book.rating;
    }

    return totalRating / this.books.length;
  },
};

bookShelf.getAverageRating(); // 7
```

Шаги: объявить переменную для суммы рейтингов → перебрать `this.books` циклом `for...of` → на каждой итерации прибавлять рейтинг книги → в конце разделить сумму на количество книг.

Изменение объекта в массиве

Частая задача — изменить свойство конкретного объекта внутри массива, найдя его по уникальному значению (например, по названию книги).

```
const bookShelf = {
  books: [
    { title: "The Last Kingdom", rating: 8 },
    { title: "The Mist", rating: 6 },
  ],
  changeRating(bookName, newRating) {
    for (const book of this.books) {
      if (book.title === bookName) {
        book.rating = newRating;
      }
    }
  },
};

bookShelf.changeRating("The Mist", 9);
bookShelf.changeRating("The Last Kingdom", 4);
```

Порядок действий:

1. Перебрать массив объектов циклом `for...of`.
2. Внутри цикла проверить, совпадает ли значение свойства текущего объекта с искомым.
3. Если совпадает — `book` в этой итерации это ссылка (reference) на нужный объект, поэтому можно сразу присвоить новое значение его свойству.

Так как объекты передаются по ссылке (by reference), изменение `book.rating` внутри цикла меняет именно тот объект, что лежит в массиве `this.books`.

Синтаксис spread и rest

Остаточные параметры (rest parameters)

Функцию можно вызывать с любым количеством аргументов (arguments), независимо от того, сколько параметров (parameters) объявлено. Лишние аргументы не вызывают ошибку — просто используются только первые по счёту:

```
function multiply(a, b) {
  console.log(a, b);
}

multiply(1, 2);           // 1 2
multiply(1, 2, 3);       // 1 2
multiply(1, 2, 3, 4);    // 1 2
```

Раньше для сбора всех аргументов использовался псевдомассив (pseudo-array) arguments:

```
function multiply() {
  console.log(arguments);
}

multiply(1, 2);           // псевдомассив [1, 2]
multiply(1, 2, 3, 4);    // псевдомассив [1, 2, 3, 4]
```

Начиная со стандарта ES6 (ECMAScript 6), появился синтаксис остаточных параметров (...rest) — он собирает все переданные значения в обычный массив (array):

```
function multiply(...args) {
  console.log(args);
}

multiply(1, 2);           // [1, 2]
multiply(1, 2, 3, 4);    // [1, 2, 3, 4]
```

Три точки ... перед именем параметра означают «собери оставшиеся аргументы в массив». Имя параметра произвольное, чаще всего args или rest.

Сбор части аргументов

...rest можно комбинировать с обычными параметрами — часть аргументов попадёт в отдельные переменные, а остальные соберутся в массив:

```
function multiply(first, second, ...args) {
  console.log(first, second, args);
}

multiply(1, 2);           // 1 2 []
multiply(1, 2, 3);       // 1 2 [3]
multiply(1, 2, 3, 4);    // 1 2 [3, 4]
```

Здесь: first получает первый аргумент, second — второй, а args — массив из всех остальных.

...rest собирает все оставшиеся аргументы, поэтому обязательно должен быть последним параметром в сигнатуре (signature) функции. Иначе — ошибка SyntaxError: Rest parameter must be last formal parameter.

```
// ❌ ошибка — rest не последний параметр
function multiply(first, ...args, second) {
}

// ❌ ошибка — rest не последний параметр
function multiply(...args, first, second) {
}
```

Оператор расширения (spread operator) — передача аргументов

Иногда нужно сделать обратное — передать массив в функцию поэлементно, как отдельные аргументы. Например, встроенная функция `Math.max()` ищет наибольшее число среди переданных аргументов и ожидает не массив, а произвольное количество отдельных чисел:

```
Math.max(14, -4, 25, 8, 11);
```

Если есть массив температур, просто передать его целиком не получится:

```
const temps = [14, -4, 25, 8, 11];

// ❌ не работает — передаём весь массив целиком
console.log(Math.max(temps)); // NaN
```

Решение — оператор расширения (spread operator, `...spread`). Он визуально похож на `rest` (тоже `...`), но делает противоположное: превращает массив в список отдельных аргументов.

```
const temps = [14, -4, 25, 8, 11];

console.log(...temps); // 14 -4 25 8 11 — набор отдельных чисел

// ✅ передаём элементы коллекции как отдельные аргументы
console.log(Math.max(...temps)); // 25
```

Spread — создание массива

Оператор расширения (`...spread`) позволяет создать копию массива или объединить несколько массивов в один. Раньше для этого использовались методы `slice()` и `concat()`, `spread` делает то же самое короче.

Копирование массива — это независимая копия, изменения одного не влияют на другой:

```
const temps = [14, -4, 25, 8, 11];

const copyOfTemps = [...temps];
console.log(copyOfTemps); // [14, -4, 25, 8, 11]
```

Объединение (concatenation) нескольких массивов в новый — порядок перечисления важен, он определяет порядок элементов в результате:

```
const lastWeekTemps = [14, 25, 11];
const currentWeekTemps = [23, 17, 18];
const allTemps = [...lastWeekTemps, ...currentWeekTemps];
console.log(allTemps); // [14, 25, 11, 23, 17, 18]
```

Spread — создание объекта

Spread также позволяет разложить свойства (properties) нескольких объектов в один новый:

```
const first = { propA: 5, propB: 10 };
const second = { propC: 15 };
const third = { ...first, ...second };
console.log(third); // { propA: 5, propB: 10, propC: 15 }
```

Порядок важен: имена свойств уникальны, поэтому если ключи совпадают — свойство, расплывлённое позже, перезапишет более раннее значение:

```
const first = { propA: 5, propB: 10, propC: 50 };
const second = { propC: 15, propD: 20 };

const third = { ...first, ...second };
console.log(third); // { propA: 5, propB: 10, propC: 15, propD: 20 }

const fourth = { ...second, ...first };
console.log(fourth); // { propA: 5, propB: 10, propC: 50, propD: 20 }
```

При распылении можно добавлять свойства в любое место записи — важно только помнить про уникальность ключа и что более позднее значение перезапишет более раннее:

```
const first = { propA: 5, propB: 10, propC: 50 };
const second = { propC: 15 };

const third = { propB: 20, ...first, ...second };
console.log(third); // { propA: 5, propB: 10, propC: 15 }

const fourth = { ...first, ...second, propB: 20 };
console.log(fourth); // { propA: 5, propB: 20, propC: 15 }
```